实验驱动学习 / 第一阶段复盘

Agent 学习讲义 01

从文件工具到双工具真实循环

概念、代码、实验、错题与复习路线

学习日期：2026 年 10 月 21 日
项目：ExperimentAgent（代码实验助手）
学习环境：Windows / Anaconda / Python 3.11 / 本地 Ollama
本讲范围：16 次小课、14 个运行实验、1 次独立功能修改
当前关卡：G1 前的工程基础；已跑通最小模型与工具闭环

本文是本项目的阶段性学习讲义，不是 CS2680 官方讲义。
实际运行、教学模拟、独立修改与后续规划分别标记。
原始源码、笔记和报告通过附录索引保留，复盘数字以对应运行记录为准。

目录

1 先建立一张完整的知识地图 1

1.1 今天已经搭出了什么 . . . 1

1.2 Chat、workflow 与 Agent . . . 1

1.3 三类证据要分开 . . . 2

2 环境与复现方式 2

2.1 使用专用 Anaconda 环境 . . . 2

2.2 本机模型信息 . . . 3

3 从普通文件读取函数开始 3

3.1 先掌握输入、动作与返回值 . . . 3

3.2 路径为什么容易出错 . . . 4

3.3 先检查授权范围，再访问环境 . . . 4

4 注册表：名称如何变成函数调用 4

4.1 字符串、函数对象和函数返回值 . . . 4

4.2 包装函数固定可信根目录 . . . 5

4.3 分发与参数展开 . . . 5

5 参数校验与 JSON Schema 5

5.1 参数校验在实际执行之前 . . . 5

5.2 strip 的准确含义 . . . 6

5.3 把参数规则描述给模型 . . . 6

6 消息协议：模型怎样提出工具请求 7

6.1 四种角色 . . . 7

6.2 工具说明不是 Python 函数本体 . . . 7

6.3 真实模型返回的请求 . . . 8

7 读取成功以后，怎样真正回传 8

7.1 保留完整 assistant 请求 . . . 8

7.2 三个时刻不能混淆 . . . 9

8 从 mock 迁移到真实自动循环 9

8.1 两种协议的差异 . . . 9
8.2 核心循环的控制流 . . . 9
8.3 为什么必须先看 tool_calls . . . 10
8.4 max_steps 计数什么 . . . 10

9 失败后，修正、重试还是停止 11

9.1 先读证据，区分层次 . . . 11
9.2 本课的失败策略 . . . 11
9.3 参数修正与原样重试 . . . 11
9.4 两个预算同时生效 . . . 12

10 Python 异常如何进入策略 12

10.1 抛异常与返回错误字典 . . . 12
10.2 三种职责 . . . 12
10.3 字段与映射 . . . 13
10.4 具体异常应先于父类捕获 . . . 13
10.5 实际异常与注入异常要分清 . . . 14

11 第二个工具：目录查看与真实选择 14

11.1 目录观察的数据结构 . . . 14
11.2 目录工具的边界 . . . 14
11.3 工具存在于两个层面 . . . 15
11.4 三种真实任务与预期轨迹 . . . 15

12 本阶段实验与结果总账 15

12.1 课程、小实验与文件对应 . . . 15
12.2 学习者已提供的真实模型结果 . . . 16
12.3 冷加载与热运行不能直接比较 . . . 17

13 测试、回归与验收范围 17

13.1 测试数量如何增长 . . . 17
13.2 两组命令都需要知道 . . . 18

13.3 对文件版本回退保持证据意识 18

14 易错点速查 19

15 自测题与参考答案 19

15.1 先不看答案，尝试复述 19

15.2 参考答案 20

15.3 已完成的独立修改 20

16 阶段结论与下次恢复入口 21

16.1 当前进度 21

16.2 下一步路线，仅作恢复提示 21

17 参考入口与证据使用说明 22

18 附录 A. 全部学习资料索引与快照 22

18.1 项目入口与环境 22

18.2 学习规划与资料 23

18.3 逐次讲义 23

18.4 运行与反馈记录 24

18.5 运行实验与夹具 24

18.6 完整 JSON 运行报告 25

18.7 Agent 实现与参数 Schema 25

18.8 项目测试 26

18.9 独立练习验收 26

18.10 学习与实验记录模板 26

18.11 学习者原始终端附件 26

19 附录 B. 关键源码：按调用链复习 27

19.1 文件读取工具 27

19.2 目录工具 27

19.3 注册、参数校验与异常转换 28

19.4 策略函数 31

19.5 原生消息自动循环 31
19.6 内部 mock 协议循环，作为对照 34
19.7 读取工具 Schema 35
19.8 目录工具 Schema 36

20 附录 C. 当前报告快照清单 36

21 附录 D. K01–K27 知识域定位 37

学习日期：2026-10-21。项目：ExperimentAgent，代码实验助手。适用环境：Windows、Anaconda、Python 3.11、本地 Ollama。

本讲覆盖本项目第 1–16 次小课、l01–l14 实验及一次独立功能修改。它是阶段复盘，不是 CS2680 官方讲义；课程讲义和论文尚未逐项阅读验收。当前主要覆盖 16 周路线前 1–3 周的工程内容，尚未整体验收 G1。

公开版说明：用户目录已用 USER 占位符脱敏，附件内部路径与无关环境名称已移除；实验结论、错误类别和耗时保留。快照清单的字节数和 SHA-256 对应公开脱敏版本。

1 先建立一张完整的知识地图

1.1 今天已经搭出了什么

现在的程序能接收任务，将工具说明交给本地模型，由模型提出工具名称与参数；Python 检查、执行并记录真实结果；模型根据观察继续行动或回答。程序具备步数上限、参数修正预算和部分文件异常的结构化处理。

当前工具只有两种：`read_text_file` 读取 UTF-8 正文；`list_directory` 查看目录的一层条目。还没有运行训练、编辑代码、持久化记忆、语义检索或实验指标验证。

```
用户目标
  → 模型读取任务、历史和工具说明
  → assistant 提出 tool_calls
  → Python 校验名称、参数与路径范围
  → 工具实际访问文件系统
  → 记录 role=tool 的真实结果
  → 策略判断继续、修正或停止
  → 再次请求模型，或者由运行时终止
```

模型提出动作，工具接触环境，运行时管理消息与执行边界。模型说“已经读取”不能替代真实读取结果；运行时返回 completed 也不自动证明任务内容正确。

1.2 Chat、workflow 与 Agent

形式	下一步由谁决定	本项目中的例子
单轮聊天	一次模型生成	l08 用一句话解释工具调用
固定工作流	程序预先安排步骤	先运行 l09，再运行 l10 回传结果

形式	下一步由谁决定	本项目中的例子
模型驱动循环	模型结合历史与观察提出动作，程序负责执行	l11 自动完成读取；l14 在两种工具间选择

一个任务即使使用了模型，也可能仍是固定流程。l14 的组合任务本身要求先看目录再读取；这验证了这一小范围内的工具选择与观察使用，不代表通用规划能力已经通过评测。

1.3 三类证据要分开

证据	能支持什么	不能替代什么
预设 mock 请求	循环在指定失败输入下行为可控	真实模型的纠错或推理能力
一次真实模型运行	该次任务中的实际调用与回答	任意任务上的可靠性结论
学习者独立修改并通过验收	这一具体分支的实现能力	从空白编写整个 Agent 的能力

2 环境与复现方式

2.1 使用专用 Anaconda 环境

在 Anaconda Prompt 中运行：

```
cd /d E:\study\agent
conda activate agent-study
python -c "import sys; print(sys.executable)"
```

已确认解释器为 E:/Anaconda3/envs/agent-study/python.exe，Python 3.11.17。Agent 学习代码目前使用 Python 标准库，没有要求安装完整 Agent 框架。

```
python -X utf8 -m labs.l01_read_file
python -X utf8 -m unittest discover -s tests -v
```

- `-X utf8`：启用 Python UTF-8 模式，便于 Windows 终端和文本处理。
- `-m labs.xxx`：从项目根目录按模块运行，便于使用项目内导入。

- `discover -s tests`：发现 tests 目录中的 unittest 测试。

`environment.yml` 记录了 python=3.11 与 pip。原有环境不需要反复重建。后续安装新依赖时应单独记录版本，不把“已安装”写成“已验证可用”。

2.2 本机模型信息

项目	已观察到的配置
本地 API	http://127.0.0.1:11434/api/chat
Ollama 版本	0.40.6，来自本机元数据记录
主实验模型	qwen3.5:9b-q4_K_M
另一个已安装模型	qwen3.5:2b
GPU	RTX 5080，总显存 16303 MiB，先前只读检查记录
当前运行接口	urllib.request 的 JSON HTTP 请求

这些是本次机器与运行的记录，不是推荐的最新软件版本。权重安装和元数据查询不等于模型推理成功；聊天成功也不等于工具调用已成功。

3 从普通文件读取函数开始

3.1 先掌握输入、动作与返回值

```
path = Path(project_root) / relative_path
content = path.read_text(encoding="utf-8")
return {"status": "success", "content": content}
```

参数告诉工具读取哪里，`read_text` 才真正访问文件。返回的字典不是模型回答，而是 Python 执行得到的工具观察。

已有文件实际内容为：

```
你好，Agent!
这是一次真实的文件读取。
```

正文的最后一个换行也属于内容；精确测试会检查它。`read_text` 的成功并不等于后续模型已经看见结果。

3.2 路径为什么容易出错

最初 train.csv 场景是纸面练习，不是已经运行过的训练。若进程 cwd 是 E:/study/agent，普通相对路径 data/train.csv 对应 E:/study/agent/data/train.csv。

如果数据实际在 labs/demo/data/train.csv，可在该 cwd 下修正相对路径，或者显式以正确根目录为锚点。我们的工具采用第二种思路：根目录由运行程序持有，模型只提供 `relative_path`。

FileNotFoundError 只表明在实际解析位置没有找到目标。还要区分数据未准备、路径写错、cwd 不符等原因；不能因为缺文件就擅自生成训练数据。

3.3 先检查授权范围，再访问环境

```
root = Path(project_root).resolve()
path = (root / relative_path).resolve()
if not path.is_relative_to(root):
    return {"status": "failed", "error_type": "path_outside_project"}
```

resolve 将路径转换为规范化路径，并解析可解析的链接；`is_relative_to` 判断它是否处于项目根目录下。不能用字符串前缀判断，例如 project-copy 与 project 可能共享文本前缀，却不是同一目录范围。

输入或环境	应发生的事
项目内存在的文件	允许访问，成功读取
项目内不存在的文件	范围允许；实际读取返回 `file_not_found`
../outside.txt	访问前拒绝，`path_outside_project`
项目外绝对路径，即使文件不存在	先拒绝越界，不先报缺文件

范围许可与目标存在是两个不同问题。当前检查不是完整操作系统沙箱，也没有解决所有并发路径变化与资源隔离问题。

4 注册表：名称如何变成函数调用

4.1 字符串、函数对象和函数返回值

```
tools = {"read_text_file": read_project_file}
tool = tools["read_text_file"]
result = tool(relative_path="labs/fixtures/hello.txt")
```

- `"read_text_file"` 是字典键，也就是公开工具名称。
- `read_project_file` 没有括号，表示函数对象。
- `read_project_file(...)` 带括号，表示现在执行函数，并取得返回值。

学习者已经正确复述：先取出函数，下一行才开始执行读取。注册时若写成函数调用，通常存进去的是那一次调用的返回值，不再是稍后可调用的函数。

4.2 包装函数固定可信根目录

```
root = str(Path(project_root).resolve())

def read_project_file(relative_path):
    return read_text_file(root, relative_path)
```

包装函数记住构造注册表时的 root。不同注册表可以绑定不同项目根目录，互不混用；模型不能额外提供 `project_root` 来改变授权范围。

4.3 分发与参数展开

```
tool = tools.get(name)
result = tool(**arguments)
```

当 arguments 为 `{"relative_path": "hello.txt"}`，`**arguments` 相当于关键字调用 `tool(relative_path="hello.txt")`。未知名称返回 `unknown_tool`；不存在的删除工具不会因模型给出名称而自动获得实现。

5 参数校验与 JSON Schema

5.1 参数校验在实际执行之前

两个路径工具现在复用 `validate_relative_path_arguments`，依次检查：

1. arguments 必须是字典。
2. 必须包含 `relative_path`。
3. 不能带额外字段，包括 `project_root`。
4. 值必须是字符串。
5. 字符串不能是空串，也不能全部为空白。

```
value = arguments["relative_path"]
if not isinstance(value, str) or not value.strip():
    return "relative_path must be a non-empty string"
```

这里利用 or 的短路求值：不是字符串时，右边不会调用 strip。校验器返回错误说明，分发器将其包装为 `invalid_arguments`；参数不合法时，工具不执行。

5.2 strip 的准确含义

strip 只移除两端空白，返回一个新字符串，不会删除中间空格，也不会原地改写输入。本项目只用 strip 的结果判断是否全空白，实际传给工具的路径保持原样。

字符串	校验是否通过	说明
空字符串	否	没有内容
三个空格	否	strip 后为空
` a`	是	包含非空白字符；实际路径仍带两端空格
`my notes.txt`	是	中间空格是文件名的一部分

5.3 把参数规则描述给模型

```
{
  "type": "object",
  "properties": {
    "relative_path": {
      "type": "string",
      "minLength": 1,
      "pattern": "\\S"
    }
  },
```

```
  "required": ["relative_path"],
  "additionalProperties": false
}
```

- minLength: 1 只检查长度；空格也计算长度。
- pattern 的正则 `\S` 表示至少出现一个非空白字符；在 JSON 源码中反斜杠需要转义。
- required 指定必填键；additionalProperties: false 描述不允许额外键。

`a` 满足这两个字符串条件。Schema 不会替我们执行 strip，也不会证明路径存在、位于项目内或具备访问权限。

Schema 是模型看到的输入契约；Python 校验负责真正执行约束。json.loads 只是解析 JSON，本项目尚未安装通用 JSON Schema 验证器。

6 消息协议：模型怎样提出工具请求

6.1 四种角色

role	本项目中的用途
system	设置助手身份及读取必须来自工具等规则
user	用户任务或用户提供的资料，不只限于问句
assistant	模型给出的文本或工具调用请求
tool	实际工具执行结果，包括成功和失败

角色说明来源与用途；不能仅根据内容是不是问题来决定 role。工具失败时依然使用 role=tool。

6.2 工具说明不是 Python 函数本体

Ollama 请求中的 tools 使用以下外层包装：

```
{
  "type": "function",
  "function": {
    "name": "read_text_file",
    "description": " 读取项目内指定 UTF-8 文件的文本内容。",
```

```
    "parameters": {"type": "object"}
  }
}
```

这里 parameters 是缩略示意，实际发送完整 Schema。工具说明不包含可执行 Python 函数，也没有自动附带文件正文。模型返回名称与参数，由本地运行时查找函数。

6.3 真实模型返回的请求

```
{
  "role": "assistant",
  "content": "",
  "tool_calls": [{
    "id": "call_example",
    "function": {
      "name": "read_text_file",
      "arguments": {"relative_path": "labs/fixtures/hello.txt"}
    }
  }]
}
```

示例 id 仅为说明；实际 id 在报告中。content 为空不一定失败，模型可能通过 `tool_calls` 提出操作。done=true、`done_reason`=stop 只表示本轮生成结束，不表示整个用户任务完成。

`tool_calls` 是列表，一轮可能有多个请求。本程序会处理完整批次，不只取第一项；执行方式仍是顺序执行，没有并发工具执行。

7 读取成功以后，怎样真正回传

7.1 保留完整 assistant 请求

```
messages.append(assistant_message)
result = call_tool(tools, name, arguments)
messages.append({
    "role": "tool",
    "tool_name": name,
    "content": json.dumps(result, ensure_ascii=False),
})
```

`assistant_message` 要保留 `tool_calls` 等完整字段，不能因为 content 为空就仅记录空文本。工具结果是字典，Ollama 此处的 content 使用字符串，因此通过 json.dumps 编码。

Python 字典、JSON 文本、包含 JSON 文本的外层消息是不同层次。打印时出现转义引号和 `\n` ，是字符串表示，并不意味着文件正文被改写。

本讲采用 Ollama 的 `tool_name` 格式；不能据此推断所有供应商都使用相同字段。调用 id 随原始 assistant 消息保留，其他服务如何关联调用要以其文档为准。

7.2 三个时刻不能混淆

```
call_tool(...)              工具已经执行
messages.append(result)     结果只保存在本地列表
chat(messages, ...)         历史真正发给模型
```

学习者已准确解释：append 不会发送网络请求，要等下一轮 chat 才把工具结果发给模型。chat 是函数对象，在本实验中的实现使用 urlopen 发起真实 HTTP 请求；测试中的 chat 则可能只是预设响应函数。

8 从 mock 迁移到真实自动循环

8.1 两种协议的差异

项目	l06 mock	原生 Ollama 循环
模型来源	手写确定性 Python 分支	本地真实模型 HTTP 响应
请求形状	type=`tool_call`、name、arguments	message.`tool_calls`
结束标记	type=final	无新工具请求，有完整非空回答
工具观察	内部教学字典	role=tool，content 为 JSON 字符串

原有 loop.py 用来保留内部协议教学示范；`ollama_loop`.py 使用原生协议。mock 的“看到错误后修正”是预写逻辑，不能当成模型自主纠错证据。

8.2 核心循环的控制流

```
messages = initial_messages
for step in range(1, max_steps + 1):
```

```
response = chat(messages, tool_descriptions)
message = response["message"]
messages.append(message)
calls = message.get("tool_calls") or []
if calls:
    # 校验、执行所有请求，保存观察，应用策略
    continue
# 没有调用且有完整非空文本，返回 completed
```

这是简化教学代码；实际实现还检查响应类型、done、`done_reason`、调用结构与网络错误，并保存诊断信息。

- messages 在循环外创建，历史才能跨轮保留。
- continue 结束当前一轮，进入下一轮请求，不清空历史。
- return 结束整个循环函数。

8.3 为什么必须先看 tool_calls

l14 组合任务第二轮实际同时包含“现在我将读取该文件的内容”和 `read_text_file` 请求。程序先执行工具，再把结果发给下一轮，不能把这段说明性文字当作最终回答。

如果删掉执行工具之后的 continue：工具可能已经执行，但随后仍会检查原 assistant 消息的 content；非空时提前返回 completed，模型没收到新观察。若 content 为空，则走到 `empty_model_response`，也会提前结束。

8.4 max_steps 计数什么

一步是一次模型请求。一次响应里执行两个工具，仍只消耗一次模型决策；最终回答也要消耗一次决策。

轨迹	模型决策数
提出读取请求 → 根据结果回答	2
错参数 → 修正参数 → 最终回答	3
查看目录 → 读取发现的文件 → 回答	3

学习者曾将 mock 的 `max_steps` 改为 2：第二次工具读取成功，但没有第三次 final 决策机会，程序正确返回 `max_steps_exceeded`。`max_steps` 是上限，不要求用完。

9 失败后，修正、重试还是停止

9.1 先读证据，区分层次

最初的实验诊断题中，status、`exit_code`、stderr 分别表示工具总结状态、子进程退出信息和诊断输出。未来执行实验时应共同记录，还要保存实际配置、cwd、种子、数据划分和代码版本。

目前读取工具没有启动训练子进程。工具返回 status=failed 后，演示脚本可能仍以退出码 0 正常结束，因为错误已被处理。不能把工具状态和外层进程退出码当作同一个字段。

9.2 本课的失败策略

工具结果	动作	理由或限制
success	continue	继续把观察交给模型
`invalid_arguments`，尚有预算	correct	允许模型根据真实错误重提参数
`invalid_arguments`，预算用完	stop	`correction_limit_exceeded`
`file_not_found`	stop	当前指定输入无法读取
`path_outside_project`	stop	超出项目范围
`unknown_tool`	stop	程序未注册该能力
未分类失败	stop	尚无证据支持继续尝试

这些规则针对当前只读任务，不是所有 Agent 的通用策略。未来若允许目录搜索，缺文件可能触发受控搜索；未知工具名也可设计成一次受限修正。当前实现没有这些恢复分支。

9.3 参数修正与原样重试

```
参数修正：path → invalid_arguments → 模型改为 relative_path
原样重试：相同请求因暂时性故障失败，等待后重新提交
```

前者改变请求内容，后者通常保持请求内容。是否适合后者，要确认错误是否可能恢复、重复执行是否安全，以及次数和时间预算是否足够。

本项目尚未实现网络自动退避、抖动或有副作用操作的幂等控制。模型请求超时仍停止，不因出现 timeout 就盲目重新提交。

9.4 两个预算同时生效

`max_steps` 限制任务总模型请求次数；`max_corrections` 默认只给一次参数修正机会，按整个任务累计，成功后不补满。

即使 `max_steps`=5 还有剩余，只要第二次参数仍错、修正机会已耗尽，就停止。学习者已经独立解释这个判断。允许一次机会不保证模型修正成功，也不保证总步数一定够完成最终回答。

10 Python 异常如何进入策略

10.1 抛异常与返回错误字典

函数正常返回 failed 字典时，调用者拿得到 result；函数抛异常而未捕获时，result 赋值没有完成，后面的消息追加和策略判断都还没有执行。

分发器只在实际调用位置捕获已知异常：

```
try:
    return tool(**arguments)
except UnicodeDecodeError as error:
    return _exception_result(name, "invalid_encoding", error)
```

except ... as error 将异常对象绑定到 error。type(error).`__name__` 获取原始类型名；str(error) 获取说明文本。辅助函数返回失败字典后，循环才可以继续处理这个观察。

10.2 三种职责

位置	负责什么
except / 异常转换辅助函数	捕获已知异常，描述发生了什么
`decide_tool_result`	根据类别和预算判断下一步动作
`run_ollama_loop`	保存结果，并执行 continue 或 return

策略返回 stop 只是一个动作决定，循环中的 return 才真正结束函数。except 本身不选择“哪一个策略函数”；本实验启用哪个策略，是调用循环时显式传入的。

10.3 字段与映射

Python 异常	error_type	当前策略 reason
FileNotFoundError	file_not_found	required_file_missing
PermissionError	permission_denied	file_access_denied
IsADirectoryError	is_a_directory	expected_file
NotADirectoryError	not_a_directory	invalid_parent_directory
UnicodeDecodeError	invalid_encoding	utf8_required
TimeoutError	tool_timeout	tool_timeout_needs_review
其他 OSError	io_error	io_failure

error_type 是程序定义的类别，供策略分支使用；exception_type 是实际 Python 类型，方便排错；message 保存细节，策略不靠文本匹配来决定动作。

已有 read_text_file 内部处理 FileNotFoundError 并直接返回原错误字典，因此不一定包含 exception_type。只有传播到分发器、在这里被捕获的异常，才增加这类诊断字段。

10.4 具体异常应先于父类捕获

```
OSError
  + FileNotFoundError
  + PermissionError
  + IsADirectoryError
  + NotADirectoryError
  + TimeoutError

ValueError
  + UnicodeError
      + UnicodeDecodeError
```

TimeoutError 是 OSError 的子类，匹配到第一个适用的 except 后就执行该分支。如果 OSError 在前，超时会被统一归为 io_error。学习者已独立增加超时分支、策略理由，并正确解释这个顺序。

没有使用 except Exception 把所有问题都压成工具失败。工具内部意外的 TypeError 等编程错误仍会暴露，不能误报成模型参数错误；KeyboardInterrupt、SystemExit 也不在本次捕获范围。

10.5 实际异常与注入异常要分清

l13 用真实非法 UTF-8 字节触发 UnicodeDecodeError；目录读取在本机 Windows 上实际触发 PermissionError；另一个权限案例是工具主动 raise PermissionError，不改 Windows ACL。

目录返回 PermissionError 不足以断言发生了 ACL 配置错误，不同操作系统可能给出不同异常。临时夹具会清理，报告中的临时路径在实验后不存在是正常现象。

11 第二个工具：目录查看与真实选择

11.1 目录观察的数据结构

```
{
  "status": "success",
  "entries": [{
    "name": "hello.txt",
    "type": "file",
    "relative_path": "labs/fixtures/hello.txt"
  }],
  "truncated": false
}
```

这只说明名称、类型和项目相对位置，没有正文。学习者最新回答正确：下一步把条目中的 `relative_path` 放进 `read_text_file` 的参数。

11.2 目录工具的边界

- 使用 iterdir 获取直接子项，不递归遍历整棵目录树。
- 先 resolve 并检查项目范围，再枚举。
- 按名称排序，最多返回 50 项，超过则 truncated=true。
- 空目录是成功的空结果，不是异常。
- 返回项目相对路径，方便接给读取工具。

- 对符号链接标记类型，后续访问仍需检查实际解析位置。

50 项仅限制返回内容，不是枚举耗时或内存的硬上限。truncated=true 时，不能断言已经知道目录里全部文件。

11.3 工具存在于两个层面

Python 注册表让程序能够执行函数；请求中的工具说明让模型知道工具名称、用途和输入规则。只增加其中一个层面，不等于完成接入。

```
{
    "read_text_file": read_project_file,
    "list_directory": list_project_directory,
}
```

两个工具共享 `relative_path` 的类型、必填与额外键校验，函数名已通用化为 `validate_relative_path_arguments`。运行时仍用模型返回的 function.name 分发，没有按任务关键词写路由规则。

11.4 三种真实任务与预期轨迹

任务	实际工具序列	模型决策数
查看目录名称和类型	`list_directory`	2
读取已知文件正文	`read_text_file`	2
先找目录中的.txt 文件，再读取	`list_directory` → `read_text_file`	3

每个任务是新会话，向模型提供相同的两份工具说明。EXPECTED 只在运行后对照这几个任务的简单序列，不影响模型选择。序列相符也不替代对回答内容的检查。

12 本阶段实验与结果总账

12.1 课程、小实验与文件对应

小实验	主要知识	对应小课	类型
l01_read_file	文件读取、UTF-8、成功与缺文件结果	02	真实文件工具
l02_path_scope	resolve、路径范围、存在与许可	03	路径观察
l03_tool_registry	名称分发、函数引用、闭包根目录	04	手工请求与真实工具
l04_tool_arguments	缺字段、错类型、空白、额外参数	05	参数错误实验
l05_tool_schema	向模型描述输入契约	06	无模型 API
l06_mock_agent_loop	观察反馈、修正、最终回答、步数	07	确定性替身与真实工具
l07_local_model_info	服务与模型元数据	08	只查询，不推理
l08_first_local_chat	第一条真实聊天请求	09	真实模型
l09_first_tool_call	原生 tool_calls，仅提出请求	10	真实模型，不执行工具
l10_tool_result_reply	执行保存请求、回传真实结果	11	真实模型与工具
l11_real_agent_loop	自动请求、执行、回传、停止	12	真实自动循环
l12_tool_failure_policy	错误分类与一次修正机会	13	预设请求与真实工具
l13_tool_exceptions	原始异常、字典转换、策略停止	14	真实文件与明确注入
超时分类独立练习	子类捕获顺序、策略映射	15	学习者独立修改
l14_multi_tool_agent	两种工具选择与观察串联	16	三个真实模型任务

第 01 小课的 train.csv 是纸面诊断；未因此宣称训练已执行。小课编号与 16 周路线的周次不是同一个维度。

12.2 学习者已提供的真实模型结果

实验	决策 / 工具观察	耗时	证据性质
l08	单轮聊天成功	51.230 秒	学习者独立成功重试
l09	1 条正确工具请求，执行数为 0	74.809 秒	学习者独立运行
l10	读取 1 次，无新工具请求，正确回答	72.302 秒	学习者独立运行
l11	决策 2 次，读取 1 次，正确回答	0.858 秒	学习者独立运行
l14 list	`list_directory`，决策 2 次	51.419 秒	学习者最新独立运行
l14 read	`read_text_file`，决策 2 次	0.856 秒	学习者最新独立运行
l14 discover	两工具依次调用，决策 3 次	1.413 秒	学习者最新独立运行

上表来自终端证据和对应报告，最后三项已核对本次附件。报告文件可能被同名的新运行覆盖，因此讲义会保存当前快照、原始终端附件与文件哈希。

12.3 冷加载与热运行不能直接比较

前一轮 AI 对 l14 三项实测耗时为 144.908、193.107、294.652 秒，多轮加载约 75–79 秒且没有输入缓存命中。学习者最新 list 的首轮加载约 31.748 秒，随后加载约 0.003 秒；read 和 discover 的模型已加载，多轮还命中输入缓存。

这解释了观测条件明显不同，但不是受控性能实验。不能据此声称代码优化了几百倍，也不能把增大 socket timeout 说成加速模型。timeout 改变允许等待的时长，不直接提高计算速度。

l08 初版曾 60 秒超时；一次 180 秒诊断请求成功约 45.028 秒。磁盘后来出现早期 60 秒版，因此要区分实际使用的版本。后续 l09–l14 的新入口独立使用 180 秒每请求等待；这是 socket timeout，不是整个任务的总截止时间。

13 测试、回归与验收范围

13.1 测试数量如何增长

阶段	tests 中的方法数	增加的主要行为
文件读取	2	已有 UTF-8 文件、项目内缺失文件

阶段	tests 中的方法数	增加的主要行为
路径范围	5	相对越界、绝对越界、范围检查顺序
注册表	9	名称、根目录绑定、分发后的路径边界
参数契约	15	类型、必填、额外键、空白、文件名空格
mock 循环	20	观察反馈、终止、非法响应、步数上限
原生循环	29	原生消息、批次、网络与截断响应
失败策略	35	修正预算、立即停止、跳过剩余工具
异常转换	41	原始异常、分类、策略停止、编程错误边界
目录工具	50	一层目录、参数、范围、截断与观察串联

另有 exercises 中 4 项超时分类验收。当前 AI 已验收项目 50 项与练习 4 项通过；学习者此前已独立提供 41 项通过输出，超时练习修改由 AI 验收。本次附件没有新增 50 项测试终端输出，不把它自动记为学习者独立复跑过。

unittest 的数量指测试方法，部分方法包含多个 subTest。50 项通过只支持这些覆盖行为，不等于覆盖所有故障，也不是 50 个真实 Agent 任务成功。

13.2 两组命令都需要知道

```
python -X utf8 -m unittest discover -s tests -v
python -X utf8 -m unittest discover -s exercises -p test_timeout_mapping.py -v
```

只运行 tests 不会执行 exercises 中的独立练习。测试中可以用预设响应稳定制造错误，但真正需要验证的文件读取、参数校验和循环控制保持真实代码。

13.3 对文件版本回退保持证据意识

学习过程中曾出现路径检查缺失、超时分支顺序变化、目录注册项回到旧版本等现象。只读检查和回归测试发现了问题，并恢复已验收的行为；来源没有确定，不能猜测是谁覆盖了文件。

复盘时应核对 sys.executable、模块 `__file__`、实际源码、文件版本以及失败测试。当前讲义附带源码快照与 SHA-256 清单，便于以后区分“当时的实现”和“后续改过的实现”。

14 易错点速查

容易混淆的说法	正确理解
注册表里只有名称字符串	键是名称字符串，值是函数对象
写函数名就是调用	加括号并传参才执行；不加括号可传递函数对象
模型生成 `tool_calls` 就已读文件	只提出请求，Python 还要校验并执行
content 为空就是失败	可能正在请求工具，要同时检查 `tool_calls`
done=true 表示任务结束	只表示本轮生成完成
append 结果就已发给模型	只是本地保存，要等下一次 chat
有一段文字就可结束	有 `tool_calls` 时先处理工具请求
`max_steps` 有剩余就继续	修正预算或明确停止规则可提前结束
except 负责决定停止策略	except 转换；策略判断；循环执行终止
捕获 TypeError 就算参数校验	校验应发生在执行前，工具内部 bug 不能误归类
目录名字已经代表文件内容	名称、类型、路径是元信息，正文还需读取
minLength=1 能排除全空格	空格有长度，需要非空白约束
strip 删除所有空格	只处理两端空白；本程序仅用它判断
文件范围检查等于完整沙箱	只实现当前路径约束，隔离与资源限制仍待学
一次热运行更快证明优化	条件不同，需区分加载、缓存和生成时间
测试通过表示整个 Agent 已可靠	只支持已覆盖行为，还需独立任务评测

15 自测题与参考答案

15.1 先不看答案，尝试复述

1. 工具说明、注册表、工具函数分别解决什么问题?
2. 给定 tools 字典，取出函数与执行函数分别是哪一行?
3. 项目外文件不存在，先返回什么错误？为什么?
4. `a` 是否通过当前字符串校验？它是否会自动变成 a?

5. 模型有 `tool_calls` 且 content 非空时，应先处理什么？
6. 文件工具成功后，为什么还需要一次模型请求？
7. append、continue、chat、return 分别做什么？
8. 第二次参数仍错，但 `max_steps` 还有剩余，如何决定？
9. `error_type`、`exception_type` 和 `stop_reason` 分别代表什么？
10. TimeoutError 放到 OSError 后有什么后果？
11. 目录列出 hello.txt 后，下一次读取使用哪个字段？
12. 为什么不能用 0.856 秒与 193.107 秒直接宣称优化效果？

15.2 参考答案

1. 工具说明告诉模型能做什么及参数契约；注册表把名称映射为函数；函数实际接触环境。
2. `tool = tools[name]` 取出对象；`tool(**arguments)` 执行。
3. `path_outside_project`。范围检查先于环境访问，不用文件是否存在来决定许可。
4. 可以通过；不会自动变成 a，本程序不改写原路径。
5. 先执行请求并保存观察，不能将说明性文字当作最终答案。
6. 工具结果尚需发送给模型，模型才能根据观察生成回答或下一步动作。
7. append 本地保存；continue 进入下一轮；本实验 chat 发网络请求；return 结束函数。
8. 修正机会已用完则停止，两个预算同时约束任务。
9. `error_type` 是程序类别，`exception_type` 是原始 Python 类型，`stop_reason` 是策略停止理由。
10. 被父类分支先接住，可能失去独立超时类别。
11. 条目里的 `relative_path`；名称、类型和路径都不包含正文。
12. 模型加载、缓存命中等条件不同，未控制变量，样本也不足。

15.3 已完成的独立修改

学习者在 registry.py 中增加了 TimeoutError 的专门分支，在 `tool_policy.py` 中增加 `tool_timeout_needs_review`；能够解释父子异常的捕获顺序。4 项练习与原有回归通过。

这是已完成的一次独立功能扩展。尚未要求从空白编写完整注册表、原生循环或第二个工具；下次可从读代码转向小范围独立实现。

16 阶段结论与下次恢复入口

16.1 当前进度

当前已经有最小 Agent 的工程原型，并具备几种已验证的失败终态。学习者能够解释核心反馈关系，并完成一次实际功能修改。第 1–3 周仍有未完成的任务清单、课程阅读和独立实现验收，不按小课数量直接推算总进度百分比。

范围	当前状态
工具读取、注册、参数与 Schema	已学习并独立运行，多处机制已复述
本地模型与原生工具循环	已独立跑通
修正预算与异常处理	已运行，超时分支已独立修改
双工具选择与目录观察接读取	最新三项独立运行成功，字段含义回答正确
整个循环与工具的独立实现	待验收
CS2680 讲义、论文逐项阅读	尚未逐项验收
检索、记忆、规划、独立评测	后续阶段
受控实验执行与真实指标验证	毕业项目下一重要阶段

16.2 下一步路线，仅作恢复提示

1. 不看答案复述“任务 → 请求 → 校验 → 工具 → 观察 → 下一轮”，并看懂两工具的消息记录。
2. 做一次由学习者亲自编写的小功能或参数规则扩展。
3. 增加项目代码定位与受控实验执行，再读取本次运行的指标。
4. 首个实验目标仍为小型合成数据线性回归：固定条件比较两个学习率，报告验证集 MSE 和耗时。该训练实验尚未创建或执行。
5. 后续依次补齐检索、记忆恢复、规划对照、独立评测、权限隔离、框架/MCP、成本优化与模型服务实验。

今天到此收束，不在本次归档中新增 Agent 功能。下次可以从本讲自测题和 PROGRESS.md 接续，不必重新安装环境或重复全部旧实验。

17 参考入口与证据使用说明

以下为本项目已保存或此前访问过的资料入口，不代表学习者已完整阅读。接口说明以项目实际采用的 Ollama 格式为例，其他服务需要适配。

- CS2680 正式课程入口
- CS2680 排行榜入口
- Hugging Face Agents Course
- Berkeley LLM Agents F24
- Berkeley Advanced LLM Agents SP25
- Ollama Chat API
- Ollama Tool Calling
- Python 3.11 pathlib
- Python 3.11 内置异常
- JSON Schema 对象规则
- JSON Schema 字符串规则
- AWS 超时、重试与退避

正文解释来自本项目实际代码、16 次小课、运行笔记、原始 JSON 报告和学习者终端证据。后附完整文件索引、关键源码快照和运行报告清单，便于复核。

18 附录 A. 全部学习资料索引与快照

本次固定保存 95 个原始文件。下列链接指向讲义目录内 `source_snapshot`，不随原项目后续编辑而变化。manifest.json 记录字节数和 SHA-256；它保证归档内容可核对，不证明课程已全部掌握。

18.1 项目入口与环境

- `.gitignore`
- `PROGRESS.md`：学习进度与验收
- `README.md`：Agent 学习与构建项目
- `environment.yml`

18.2 学习规划与资料

- `docs/01-roadmap.md`：16 周学习路线
- `docs/02-knowledge-map.md`：Agent 知识点覆盖矩阵
- `docs/03-resources.md`：课程与资料导航
- `docs/04-labs.md`：递进实验任务书
- `docs/05-capstone.md`：毕业项目：ExperimentAgent 代码实验助手
- `docs/06-learning-workflow.md`：学习方法与环境

18.3 逐次讲义

- `lessons/01-agent-loop.md`：第 1 课：代码实验 Agent 如何工作
- `lessons/02-first-file-tool.md`：第 2 次动手：文件读取工具与 Anaconda
- `lessons/03-path-scope.md`：第 3 次动手：工具可以读哪些路径
- `lessons/04-tool-registry.md`：第 4 次动手：根据工具名称调用函数
- `lessons/05-tool-arguments.md`：第 5 次动手：调用工具前检查参数
- `lessons/06-tool-json-schema.md`：第 6 次动手：用 JSON Schema 描述工具参数
- `lessons/07-mock-agent-loop.md`：第 7 次动手：把工具接进反馈循环
- `lessons/08-model-backend.md`：第 8 次动手：选择真实模型接口
- `lessons/09-first-local-chat.md`：第 9 次动手：第一次真实模型请求
- `lessons/10-real-tool-call.md`：第 10 次动手：观察真实模型的工具调用请求
- `lessons/11-tool-result-reply.md`：第 11 次动手：执行工具并回传结果
- `lessons/12-real-agent-loop.md`：第 12 次动手：真实模型的自动循环
- `lessons/13-tool-failure-policy.md`：第 13 次动手：工具失败后，修正还是停止
- `lessons/14-tool-exceptions.md`：第 14 次动手：把 Python 异常转换成工具结果
- `lessons/15-timeout-mapping-exercise.md`：第 15 次动手：由你完成工具超时分类
- `lessons/16-directory-tool-and-selection.md`：第 16 次动手：第二个工具与模型选择
- `lessons/week01.md`：第 1 周：理解代码实验 Agent，并定义任务

18.4 运行与反馈记录

- notes/2026-10-21-review.md：2026-10-21 阶段复盘
- notes/argument-validation-run.md：参数校验实操与反馈
- notes/directory-tools-run.md：查看目录与双工具选择实测
- notes/file-tool-run.md：文件读取工具实测记录
- notes/first-local-chat-run.md：第一次本地模型聊天实测
- notes/first-native-tool-call.md：真实模型首次原生工具请求
- notes/lesson01-feedback.md：第 1 课：首次练习记录
- notes/local-chat-timeout.md：本地聊天超时诊断与修正
- notes/mock-loop-run.md：确定性模型替身闭环首次运行
- notes/path-scope-run.md：路径范围检查：运行与反馈
- notes/real-agent-loop-run.md：真实模型自动循环实测
- notes/timeout-mapping-exercise.md：独立修改练习准备记录
- notes/tool-exceptions-run.md：工具异常转换实测
- notes/tool-failure-policy-run.md：工具失败策略实验记录
- notes/tool-registry-run.md：工具注册表首次示范与运行
- notes/tool-result-reply-run.md：执行真实工具请求并回传结果
- notes/tool-schema-run.md：工具参数 JSON Schema 首次演示

18.5 运行实验与夹具

- labs/__init__.py
- labs/fixtures/hello.txt
- labs/l01_read_file.py
- labs/l02_path_scope.py
- labs/l03_tool_registry.py
- labs/l04_tool_arguments.py
- labs/l05_tool_schema.py
- labs/l06_mock_agent_loop.py

- labs/l07_local_model_info.py
- labs/l08_first_local_chat.py
- labs/l09_first_tool_call.py
- labs/l10_tool_result_reply.py
- labs/l11_real_agent_loop.py
- labs/l12_tool_failure_policy.py
- labs/l13_tool_exceptions.py
- labs/l14_multi_tool_agent.py

18.6 完整 JSON 运行报告

- reports/l08_first_local_chat.json
- reports/l08_initial_success.json
- reports/l08_timeout_diagnostic.json
- reports/l09_first_tool_call.json
- reports/l10_tool_result_reply.json
- reports/l11_real_agent_loop_steps1.json
- reports/l11_real_agent_loop_steps5.json
- reports/l12_tool_failure_policy.json
- reports/l13_tool_exceptions.json
- reports/l14_multi_tool_agent_discover.json
- reports/l14_multi_tool_agent_list.json
- reports/l14_multi_tool_agent_read.json

18.7 Agent 实现与参数 Schema

- experiment_agent/__init__.py
- experiment_agent/loop.py
- experiment_agent/ollama_loop.py
- experiment_agent/tool_policy.py

- experiment_agent/tools/__init__.py
- experiment_agent/tools/directory_tools.py
- experiment_agent/tools/file_tools.py
- experiment_agent/tools/list_directory.schema.json
- experiment_agent/tools/read_text_file.schema.json
- experiment_agent/tools/registry.py

18.8 项目测试

- tests/test_directory_tools.py
- tests/test_file_tools.py
- tests/test_loop.py
- tests/test_ollama_loop.py
- tests/test_tool_exceptions.py
- tests/test_tool_failure_policy.py
- tests/test_tool_registry.py

18.9 独立练习验收

- exercises/test_timeout_mapping.py

18.10 学习与实验记录模板

- templates/experiment-report.md：实验报告：< 编号与标题 >
- templates/study-note.md：学习记录：< 主题 >

18.11 学习者原始终端附件

- learner_evidence/learner_l11_output.txt
- learner_evidence/learner_l12_output.txt
- learner_evidence/learner_l14_output.txt

19 附录 B. 关键源码：按调用链复习

以下摘录是本次归档时的真实源码，不是额外实现的新版本。完整实验与测试见附录 A。

19.1 文件读取工具

来源：`experiment_agent/tools/file_tools.py`。

```
""" 文件读取工具。"""

from pathlib import Path


def read_text_file(project_root: str, relative_path: str) -> dict:
    """ 仅尝试读取指定项目范围内的 UTF-8 文本。"""
    root = Path(project_root).resolve()
    path = (root / relative_path).resolve()

    if not path.is_relative_to(root):
        return {
            "status": "failed",
            "error_type": "path_outside_project",
            "path": str(path),
        }

    try:
        content = path.read_text(encoding="utf-8")
        return {
            "status": "success",
            "content": content,
        }
    except FileNotFoundError:
        return {
            "status": "failed",
            "error_type": "file_not_found",
            "path": str(path),
        }
```

19.2 目录工具

来源：`experiment_agent/tools/directory_tools.py`。

```
""" 项目范围内的一层目录查看工具。"""

from pathlib import Path


def list_directory(project_root: str, relative_path: str) -> dict:
    """ 列出直接子项，最多返回 50 项；文件正文由读取工具获取。"""
    root = Path(project_root).resolve()
    path = (root / relative_path).resolve()
    if not path.is_relative_to(root):
        return {"status": "failed", "error_type": "path_outside_project", "path":
        ↪  str(path)}

    children = sorted(path.iterdir(), key=lambda child: child.name.casefold())
    entries = []
    for child in children[:50]:
        if child.is_symlink():
            entry_type = "symlink"
        elif child.is_dir():
            entry_type = "directory"
        elif child.is_file():
            entry_type = "file"
        else:
            entry_type = "other"
        entries.append({
            "name": child.name,
            "type": entry_type,
            "relative_path": child.relative_to(root).as_posix(),
        })
    return {"status": "success", "entries": entries, "truncated": len(children) >
    ↪  50}
```

19.3 注册、参数校验与异常转换

来源：`experiment_agent/tools/registry.py`。

```
""" 工具名称与可调用函数的映射。"""

from pathlib import Path
```

```
from experiment_agent.tools.directory_tools import list_directory
from experiment_agent.tools.file_tools import read_text_file


def build_tool_registry(project_root: str) -> dict:
    """ 创建绑定到指定项目根目录的工具表。"""
    root = str(Path(project_root).resolve())

    def read_project_file(relative_path: str) -> dict:
        return read_text_file(root, relative_path)

    def list_project_directory(relative_path: str) -> dict:
        return list_directory(root, relative_path)

    return {
        "read_text_file": read_project_file,
        "list_directory": list_project_directory,
    }


def validate_relative_path_arguments(arguments: object) -> str | None:
    """ 参数合法时返回 None，否则返回错误说明。"""
    if not isinstance(arguments, dict):
        return "arguments must be a dictionary"

    if "relative_path" not in arguments:
        return "relative_path is required"

    if set(arguments) != {"relative_path"}:
        return "only relative_path is allowed"

    value = arguments["relative_path"]
    if not isinstance(value, str) or not value.strip():
        return "relative_path must be a non-empty string"

    return None


def _exception_result(name: str, error_type: str, error: Exception) -> dict:
    """ 把已知执行异常转换成可记录、可交给策略判断的结果。"""
    return {
        "status": "failed",
        "error_type": error_type,
        "tool_name": name,
```

```
        "exception_type": type(error).__name__,
        "message": str(error),
    }


def call_tool(tools: dict, name: str, arguments: object) -> dict:
    """ 按名称查找并校验参数，在实际执行边界转换已知文件异常。"""
    tool = tools.get(name)

    if tool is None:
        return {
            "status": "failed",
            "error_type": "unknown_tool",
            "tool_name": name,
        }

    if name in {"read_text_file", "list_directory"}:
        message = validate_relative_path_arguments(arguments)
        if message is not None:
            return {
                "status": "failed",
                "error_type": "invalid_arguments",
                "message": message,
            }

    try:
        return tool(**arguments)
    except FileNotFoundError as error:
        return _exception_result(name, "file_not_found", error)
    except PermissionError as error:
        return _exception_result(name, "permission_denied", error)
    except IsADirectoryError as error:
        return _exception_result(name, "is_a_directory", error)
    except NotADirectoryError as error:
        return _exception_result(name, "not_a_directory", error)
    except UnicodeDecodeError as error:
        return _exception_result(name, "invalid_encoding", error)
    except TimeoutError as error:
        return _exception_result(name, "tool_timeout", error)
    except OSError as error:
        return _exception_result(name, "io_error", error)
```

19.4 策略函数

来源：experiment_agent/tool_policy.py。

```
""" 本课针对只读文件任务的工具结果策略。"""


def decide_tool_result(result, corrections_used, max_corrections=1):
    """ 返回 continue、correct 或 stop；修正预算按整个任务累计。"""
    if not isinstance(result, dict) or result.get("status") not in {"success",
    ↪  "failed"}:
        return {"action": "stop", "reason": "invalid_tool_result"}
    if result["status"] == "success":
        return {"action": "continue", "reason": "tool_succeeded"}

    error_type = result.get("error_type")
    if error_type == "invalid_arguments":
        if corrections_used < max_corrections:
            return {"action": "correct", "reason": "ask_model_to_fix_arguments"}
        return {"action": "stop", "reason": "correction_limit_exceeded"}

    reasons = {
        "file_not_found": "required_file_missing",
        "path_outside_project": "permission_boundary",
        "unknown_tool": "tool_unavailable",
        "permission_denied": "file_access_denied",
        "is_a_directory": "expected_file",
        "not_a_directory": "invalid_parent_directory",
        "invalid_encoding": "utf8_required",
        "tool_timeout": "tool_timeout_needs_review",
        "io_error": "io_failure",

    }
    return {"action": "stop", "reason": reasons.get(error_type,
    ↪  "unclassified_tool_failure")}
```

19.5 原生消息自动循环

来源：experiment_agent/ollama_loop.py。

```
""" 使用 Ollama 原生消息格式的模型—工具循环。"""
```

```
import json

from experiment_agent.tools.registry import call_tool


def run_ollama_loop(task, chat, tools, tool_descriptions, max_steps=5,
↪  tool_policy=None):
    """chat 返回原生响应；可选 tool_policy 决定工具结果出现后继续、修正或停止。"""
    if max_steps < 1:
        raise ValueError("max_steps must be at least 1")
    messages = [
        {"role": "system", "content": "你是代码实验助手。
        ↪  读取项目文件必须通过提供的工具获得实际内容。"},
        {"role": "user", "content": task},
    ]
    responses = []
    tool_decisions = []
    corrections_used = 0

    def finish(status, steps, **fields):
        return {
            "status": status, "steps": steps, "messages": messages,
            "responses": responses, "tool_decisions": tool_decisions, **fields,
        }

    # 一步是一次模型请求；一条响应里的多个工具调用仍属于这一步。
    for step in range(1, max_steps + 1):
        try:
            response = chat(messages, tool_descriptions)
        except (OSError, json.JSONDecodeError) as error:
            return finish("failed", step, error_type="model_request_failed",
            ↪  message=str(error))
        responses.append(response)

        if not isinstance(response, dict):
            return finish("failed", step, error_type="invalid_model_response")
        if response.get("done") is not True or response.get("done_reason") !=
        ↪  "stop":
            return finish("failed", step, error_type="incomplete_model_response")
        message = response.get("message")
        if (
            not isinstance(message, dict)
            or message.get("role") != "assistant"
            or not isinstance(message.get("content", ""), str)
```

```
        ):
            return finish("failed", step, error_type="invalid_model_response")
        calls = message.get("tool_calls")
        if calls is None:
            calls = []
        if not isinstance(calls, list):
            return finish("failed", step, error_type="invalid_model_response")
        # 先检查整批调用结构，避免处理到一半才发现后一条结构不完整。
        for call in calls:
            function = call.get("function") if isinstance(call, dict) else None
            if (
                not isinstance(function, dict)
                or not isinstance(function.get("name"), str)
                or "arguments" not in function
            ):
                return finish("failed", step, error_type="invalid_model_response")

        # 保留原生 assistant 消息，包括调用 id、名称和参数。
        messages.append(message)
        if calls:
            for index, call in enumerate(calls):
                function = call["function"]
                result = call_tool(tools, function["name"], function["arguments"])
                messages.append({
                    "role": "tool",
                    "tool_name": function["name"],
                    "content": json.dumps(result, ensure_ascii=False),
                })
                if tool_policy is not None:
                    decision = tool_policy(result, corrections_used)
                    if decision["action"] == "correct":
                        corrections_used += 1
                    tool_decisions.append({
                        "step": step,
                        "tool_name": function["name"],
                        **decision,
                        "corrections_used": corrections_used,
                    })
                    if decision["action"] == "stop":
                        return finish(
                            "failed", step,
                            error_type="tool_policy_stopped",
                            tool_error_type=result.get("error_type"),
                            stop_reason=decision["reason"],
```

```
                            pending_tool_calls=len(calls) - index - 1,
                        )
                continue  # 带上真实工具结果，进入下一次模型请求。

            content = message.get("content", "")
            if content.strip():
                return finish("completed", step, answer=content)
            return finish("failed", step, error_type="empty_model_response")

    return finish("failed", max_steps, error_type="max_steps_exceeded")
```

19.6 内部 mock 协议循环，作为对照

来源：experiment_agent/loop.py。

```
""" 通过消息历史连接模型决策和工具执行。"""

from experiment_agent.tools.registry import call_tool


def run_loop(task: str, model, tools: dict, tool_descriptions: list, max_steps:
↪  int = 5) -> dict:
    """ 运行内部教学协议的模型—工具循环。"""
    if max_steps < 1:
        raise ValueError("max_steps must be at least 1")

    messages = [{"role": "user", "content": task}]

    for step in range(1, max_steps + 1):
        action = model(messages, tool_descriptions)
        messages.append({"role": "assistant", "content": action})

        if isinstance(action, dict):
            if action.get("type") == "final" and isinstance(action.get("content"),
            ↪  str):
                return {
                    "status": "completed",
                    "answer": action["content"],
                    "steps": step,
                    "messages": messages,
                }
```

```
            if (
                action.get("type") == "tool_call"
                and isinstance(action.get("name"), str)
                and "arguments" in action
            ):
                result = call_tool(tools, action["name"], action["arguments"])
                messages.append({
                    "role": "tool",
                    "name": action["name"],
                    "content": result,
                })
                continue

        return {
            "status": "failed",
            "error_type": "invalid_model_response",
            "steps": step,
            "messages": messages,
        }

    return {
        "status": "failed",
        "error_type": "max_steps_exceeded",
        "steps": max_steps,
        "messages": messages,
    }
```

19.7 读取工具 Schema

来源：experiment_agent/tools/read_text_file.schema.json。

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "relative_path": {
      "type": "string",
      "description": "相对于项目根目录的 UTF-8 文本文件路径，例如
      ↪  labs/fixtures/hello.txt。路径须位于项目内，且不能是空字符串或全空白。",
      "minLength": 1,
      "pattern": "\\S"
    }
```

```
  },
  "required": ["relative_path"],
  "additionalProperties": false
}
```

19.8 目录工具 Schema

来源：experiment_agent/tools/list_directory.schema.json。

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "relative_path": {
      "type": "string",
      "description": "相对于项目根目录的目录路径。使用 . 查看根目录，例如
      ↪  labs/fixtures 查看该子目录。路径须位于项目内，且不能是空字符串或全空白。",
      "minLength": 1,
      "pattern": "\\S"
    }
  },
  "required": ["relative_path"],
  "additionalProperties": false
}
```

20 附录 C. 当前报告快照清单

同名报告代表快照时的最近保存结果，不一定是首次运行。上文已将关键结果的学习者/AI来源分别注明。原始 JSON 保留模型响应、消息历史或实验案例；需要精确核对时打开原文件。

报告	结构与用途	时长或案例数
l08_first_local_chat.json	模型响应 done=True	51.230 秒
l08_initial_success.json	模型响应 done=True	50.340 秒

报告	结构与用途	时长或案例数
l08_timeout_diagnostic.json	模型响应 done=True	45.028 秒
l09_first_tool_call.json	模型响应 done=True	74.809 秒
l10_tool_result_reply.json	模型响应 done=True	72.302 秒
l11_real_agent_loop_steps1.json	循环终态 failed	0.522 秒
l11_real_agent_loop_steps5.json	循环终态 completed	0.858 秒
l12_tool_failure_policy.json	预设请求/异常演示的逐案例结果	5 个案例
l13_tool_exceptions.json	预设请求/异常演示的逐案例结果	4 个案例
l14_multi_tool_agent_discover.json	循环终态 completed	1.413 秒
l14_multi_tool_agent_list.json	循环终态 completed	51.419 秒
l14_multi_tool_agent_read.json	循环终态 completed	0.856 秒

21 附录 D. K01–K27 知识域定位

此处只标记本阶段覆盖程度；详细内容和未来验收标准见源文件 docs/02-knowledge-map.md。局部接触不代表整域完成。

编号	知识域	本阶段定位
K01	Agent 基本模型	基础闭环已学习，独立完整实现待验收
K02	LLM 与推理	接触生成参数与接口，模型原理未系统复习
K03	模型接入	本地聊天、原生工具消息已独立运行
K04	Prompt 与上下文	工具说明与消息历史局部接触

编号	知识域	本阶段定位
K05	工具设计	工具契约已运行，超时分支独立修改
K06	运行时控制	循环、停止、修正预算已实践；恢复/幂等未做
K07	检索与知识	后续学习，尚未验收
K08	Agentic RAG	后续学习，尚未验收
K09	记忆	后续学习，尚未验收
K10	规划与推理	后续学习，尚未验收
K11	多 Agent	后续学习，尚未验收
K12	协议与框架	后续学习，尚未验收
K13	协议扩展	后续学习，尚未验收
K14	评测设计	已有单元测试与简单轨迹核对；独立任务评测未做
K15	实验方法	已区分冷/热观测；尚无受控优化实验
K16	可观测性	保存 JSON 报告与错误；完整追踪系统未做
K17	安全与权限	已有路径边界；更完整权限和注入实验未做
K18	执行隔离	后续学习，尚未验收
K19	人机协作	后续学习，尚未验收
K20	应用工程	专用环境与模块运行已完成；产品化未做
K21	Agent 经济性	接触调用次数与时间，经济性实验未做
K22	系统基础	后续学习，尚未验收
K23	模型服务	观察加载与缓存指标；系统服务实验未做
K24	服务优化	后续学习，尚未验收
K25	学习型 Agent	后续学习，尚未验收
K26	自改进系统	后续学习，尚未验收
K27	特定应用	选定代码实验 Agent，尚未执行训练任务